

SPROCKET MOD KIT · GUIDE

Making Mods for Sprocket

How Sprocket mods work, and how to build, test and share your own, step by step: from a first "hello" mod to patching the game, drawing your own panels, and the traps we fell into so you don't have to.

Written for: Sprocket 0.2.55.5 (Unity 6000.3), BepInEx 6 IL2CPP, .NET 8 SDK

Based on: the Sprocket Mod Kit, the Quality of Life mod and the Battle Editor mod (real code, real bugs)

Kit: github.com/Hans21223/Sprocket-Quality-of-Life (MIT licence)

Date: September 2026. Game updates can change class names; the method in this guide still applies.

Contents

1	How Sprocket mods work	Data mods, code mods, the loader chain
2	Setting up your tools	Loader, .NET SDK, editor, the kit
3	Your first mod	Build, install, see it run
4	Exploring the game's code	Finding the class you need
5	Changing the game with Harmony patches	Prefix, postfix, and when hooks don't fire
6	Running your own code every frame	MonoBehaviours, input, time
7	Working with game objects (IL2CPP rules)	Casts, lists, delegates, missing methods
8	Adding user interface	Game panels, menus, overlays, the mouse
9	Settings, files and logs	Config, saving data, tracing
10	Data mods: new parts without code	Part files, names, blueprints
11	Testing and debugging	The loop, logs, offline tests
12	Packaging and sharing	Zip layout, versions, licence
13	Pitfalls we hit, and the fixes	Symptom → cause → fix
14	Case study: how the Battle Editor is built	A whole mod, file by file
A	Cheat sheet	Snippets to copy
B	Troubleshooting	"My mod doesn't..."

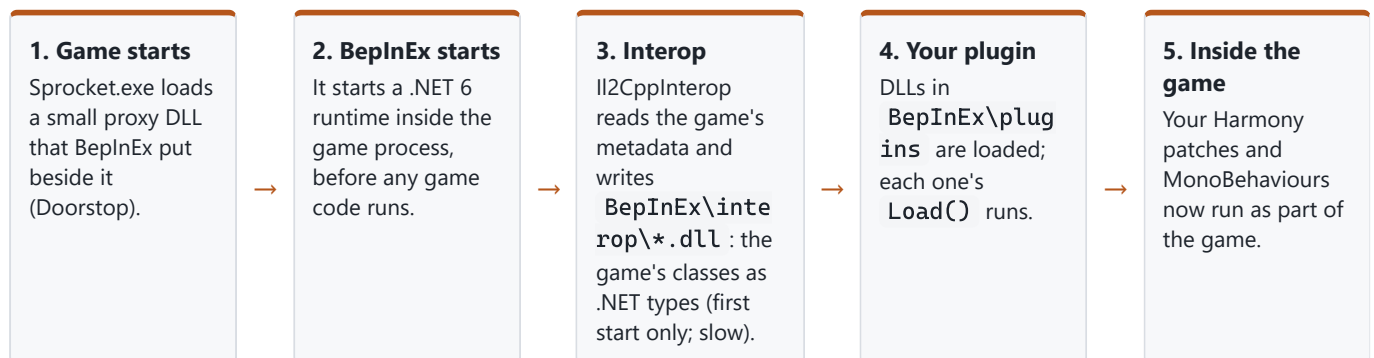
1 How Sprocket mods work

A mod is anything that changes the game without the game's makers shipping it. Sprocket has two very different kinds, and it's worth knowing which one you need before you start.

Kind	What it is	Needs
Data mod	Extra or changed game files: part definitions (JSON), their names (XML), and so on. The game reads them like its own. Example: the kit's <i>Round Add-on Parts</i> , which adds 16- and 32-sided add-ons to the editor.	Nothing extra. Works on an unmodded game.
Code mod (plugin)	A .NET DLL with your own C# code, loaded into the running game. It can read and change anything the game does: add buttons, tools, whole editors. Examples: <i>Quality of Life</i> , <i>Battle Editor</i> .	A mod loader (BepInEx 6 IL2CPP), the .NET SDK to build.

Why code mods need a loader

Sprocket is a Unity game built with **IL2CPP**: the game's C# code was turned into C++ and compiled into one native file, `GameAssembly.dll`. There are no normal .NET DLLs of the game to reference or edit. A loader bridges that gap: it starts before the game, puts a .NET runtime inside it, builds .NET "stand-in" classes for every game class, and then loads your plugin. Your code calls those stand-ins, and each call is passed through to the real game code.



Nothing on disk is changed: `GameAssembly.dll` stays exactly as Steam installed it. Remove the plugin DLL (or BepInEx) and the game is vanilla again. That is the rule for every mod in this guide: **change the running game, never the game's files.**

The words you'll see

Term	Meaning
IL2CPP	Unity's way of compiling C# to native code. Why the game's classes are "stand-ins" in your code.
BepInEx	The mod loader. The IL2CPP edition (version 6) is the one Sprocket needs.
Interop assemblies	The .NET files in <code>BepInEx\interop</code> . You compile against them. They contain every class and method signature, but no real code: calls go to the game.
Plugin	Your mod: a DLL with a class marked <code>[BepInPlugin]</code> .
Harmony	The library that lets your code run before or after any game method (a <i>patch</i>). Comes with BepInEx.

Prefix / Postfix	A patch that runs before / after the game's method. A prefix can also skip the method.
MonoBehaviour	A Unity script: its <code>Update()</code> runs every frame, <code>OnGUI()</code> draws on screen.
HDRP	Unity's High Definition Render Pipeline, which Sprocket uses. Matters for materials, lights and cameras.

2 Setting up your tools

What to install

- 1 **Sprocket** from Steam. For modding, work on a **copy of the game folder** (a "test copy"): your normal game stays clean, and a broken mod can't spoil your evening of playing.
- 2 **BepInEx 6, IL2CPP edition** (bleeding-edge builds at `builds.bepinex.dev`). Extract it into the game folder, so the `BepInEx` folder sits next to `Sprocket.exe`. Start the game once and wait: the first start builds `BepInEx\interop` and takes several minutes. Close the game when the main menu appears.

Check the loader reaches the main menu

Not every BepInEx build runs every Unity version. On Sprocket 0.2.55.5 (Unity 6000.3.21) the official build `be.788` froze at the main menu. Use a loader that other Sprocket modders confirm works on your game version, and check the menu appears before you write any code.
- 3 **.NET 8 SDK** from `dotnet.microsoft.com/download`. (Mods target `net6.0`, which the .NET 8 SDK builds.) Check it with `dotnet --version`.
- 4 **A code editor**: Visual Studio 2022 Community, VS Code with the C# Dev Kit, or Rider. Any of them gives you autocomplete on the game's classes once the project builds.
- 5 **The Sprocket Mod Kit** (the Quality of Life repository): `SprocketMod.props`, a starter mod, a class lister and a deploy script. Download it or `git clone` it.
- 6 *Optional*: **UnityExplorer** (a BepInEx plugin that lets you browse the live game's objects and components) and **Sprocket Mod Manager** (turns mods on and off without copying files by hand).

The game folder after installing BepInEx

Path (inside the Sprocket folder)	What's there
<code>BepInEx\core\</code>	BepInEx itself, Harmony, IL2CpplInterop. You compile against these.
<code>BepInEx\interop\</code>	The game's classes as .NET files (made on first start). You compile against these too.
<code>BepInEx\plugins\</code>	Where mods go. Each <code>.dll</code> here is loaded at start.
<code>BepInEx\config\</code>	Settings files, one per mod (<code><mod id>.cfg</code>).
<code>BepInEx\LogOutput.log</code>	The loader's log, including every line your mod logs. Your first stop when anything goes wrong.
<code>Sprocket_Data\StreamingAssets\</code>	The game's data files (parts, localisation). Data mods add files here.

<code>%USERPROFILE%\AppData\LocalLow\HD\Sprocket\Player.log</code>	Unity's own log (outside the game folder): scene changes, game warnings and errors.
<code>Documents\My Games\Sprocket\</code>	Your designs (<code>Factions\<faction>\Blueprints\Vehicles*.blueprint</code>) and other player files.

3 Your first mod

In this chapter you build a mod, put it in the game and see it load. After that, every other chapter is "the same, with more code".

Step 1: make the project

Copy the kit's `MyFirstMod` folder to a new folder, for example `HelloSprocket` , and rename the `.csproj` inside to `HelloSprocket.csproj` . The project file is tiny, because the shared settings live in the kit's `SprocketMod.props` :

`HelloSprocket\HelloSprocket.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">
  <Import Project="..\SprocketMod.props" />
  <PropertyGroup>
    <AssemblyName>HelloSprocket</AssemblyName>
  </PropertyGroup>
</Project>
```

What SprocketMod.props does

Setting	Why
<code>TargetFramework net6.0</code>	BepInEx 6 IL2CPP runs mods on .NET 6.
<code>GameDir</code>	Your Sprocket folder (default: the Steam folder). Change it for a test copy: <code>dotnet build -p:GameDir="D:\Games\Sprocket"</code> .
References to <code>BepInEx\core*.dll</code> and <code>BepInEx\interop*.dll</code> with <code>Private=false</code>	You compile against the game and the loader, but don't copy them into your mod: they're already in every player's game.
<code>PathMap</code>	Stops your own folder path (and your Windows user name) being written into the DLL.
A check target	Stops the build with a clear message if <code>GameDir</code> has no BepInEx or no interop folder yet.

Step 2: the plugin class

`HelloSprocket\Plugin.cs` (the smallest possible mod)

```
using BepInEx;
using BepInEx.Unity.IL2CPP;

namespace HelloSprocket;

[BepInPlugin("yourname.sprocket.hello", "Hello Sprocket", "1.0.0")] // id, name, version
public sealed class Plugin : BasePlugin
```

```
{
    public override void Load() => Log.LogInfo("Hello Sprocket loaded!");
}
```

- The **id** (`yourname.sprocket.hello`) must be unique and must **never change** between versions, or players end up with two copies. The **name** is what people see; the **version** goes up with each release.
- `Load()` runs once, while the game is starting. There's no game world yet (no tanks, no menus), so here you only set things up: settings, patches, components.

Step 3: build it

```
cd HelloSprocket
dotnet build -c Release
```

The mod is `HelloSprocket\bin\Release\net6.0\HelloSprocket.dll` . From the kit folder, `.\tools\deploy.ps1 HelloSprocket` builds and copies it into the game (add `-GameDir "..."` for a test copy).

Step 4: install and run

- **By hand:** copy the DLL into `Sprocket\BepInEx\plugins\` .
- **With Mod Manager:** the mod goes in `ModManager\mods<game>\<mod name>\` laid out like the game folder (`... \Hello Sprocket\BepInEx\plugins\HelloSprocket.dll`), then you enable it in the manager.

Close the game first

A running game locks every mod DLL, so copying over it fails. (The kit's deploy script waits for the game to close.)

Start the game and open `BepInEx\LogOutput.log` . You should find:

```
[Info : BepInEx] Loading [Hello Sprocket 1.0.0]
[Info :Hello Sprocket] Hello Sprocket loaded!
```

That's a working mod. Everything else builds on these four steps.

Step 5: a mod that does something

This version shows all three building blocks at once: a **setting**, a **component** that runs every frame (press F10 to show a message) and a **Harmony patch** (a "Say hello" button added to the menu, under Custom Battle). It is compiled and checked against Sprocket 0.2.55.5.

`HelloSprocket\Plugin.cs` (full example)

```
using BepInEx; using BepInEx.Configuration; using BepInEx.Logging; using BepInEx.Unity.IL2CPP;
using HarmonyLib; using Il2CppInterop.Runtime; using Sprocket;
using UnityEngine; using UnityEngine.Events; using UnityEngine.InputSystem;

namespace HelloSprocket;

[BepInPlugin("yourname.sprocket.hello", "Hello Sprocket", "1.0.0")]
public sealed class Plugin : BasePlugin
{
    internal static ManualLogSource ModLog = null!;
    internal static ConfigEntry<string> Greeting = null!;

    public override void Load()
    {
```

```

    ModLog = Log;
    // A setting the player can change in BepInEx\config\yourname.sprocket.hello.cfg
    Greeting = Config.Bind("General", "Greeting", "Hello from my first mod!",
        "Text shown by F10 and the menu button.");
    // Code that runs every frame lives in a MonoBehaviour; AddComponent registers it and starts it.
    AddComponent<Overlay>();
    // Harmony patches: one try per patch class, so one failure can't stop the rest.
    try { new Harmony("yourname.sprocket.hello").PatchAll(typeof(MenuButton)); }
    catch (Exception ex) { Log.LogError($"Could not attach to the main menu: {ex}"); }
    Log.LogInfo("Hello Sprocket loaded: press F10 in game.");
}
}

public sealed class Overlay : MonoBehaviour
{
    public Overlay(IntPtr pointer) : base(pointer) { } // every injected MonoBehaviour needs this

    internal static float ShowUntil;

    public void Update()
    {
        var keys = Keyboard.current; // the game uses Unity's Input System
        if (keys != null && keys.f10Key.wasPressedThisFrame) ShowUntil = Time.unscaledTime + 3;
    }

    public void OnGUI()
    {
        if (Time.unscaledTime < ShowUntil)
            GUI.Box(new Rect((Screen.width - 500) / 2f, 20, 500, 30), Plugin.Greeting.Value);
    }
}

[HarmonyPatch]
static class MenuButton
{
    static UnityAction? onClick; // kept in a field: the game holds on to it

    [HarmonyPostfix, HarmonyPatch(typeof(MenuPanel), nameof(MenuPanel.Button))]
    static void Added(MenuPanel __instance, string label, bool interactable)
    {
        try
        {
            if (label == null) return;
            if (label.IndexOf("custom battle", StringComparison.OrdinalIgnoreCase) < 0) return;
            onClick ??= DelegateSupport.ConvertDelegate<UnityAction>(
                new Action(() => Overlay.ShowUntil = Time.unscaledTime + 3));
            __instance.Button("Say hello", onClick, interactable);
        }
        catch (Exception ex) { Plugin.ModLog.LogError($"Menu button: {ex}"); } // never reaches the game
    }
}

```

Chapters 5–8 explain each part of this in detail.

4 Exploring the game's code

Most modding time is spent finding out *which* game class does the thing you want to change. Here's how.

The interop files are your map

Every game class is in `BepInEx\interop` with its fields, properties and methods. These files hold **signatures only**: the method bodies are stubs that call into the native game. So they tell you *what exists and what it's called*, not *how it works*. You learn how it works by trying things and logging what happens.

List classes with the kit's ApiLister

```
# classes whose name contains "Turret"
dotnet run --project tools\ApiLister -- "C:\...\Sprocket\BepInEx\interop" ~Turret
# one class: its base class, fields and methods
dotnet run --project tools\ApiLister -- "C:\...\Sprocket\BepInEx\interop" TurretRingEditor
```

It only reads the files; no game code runs. Any .NET decompiler (ILSpy, dnSpy) can also open the interop DLLs if you prefer browsing.

Reading what you find

- Game classes appear twice, as `Sprocket.X` and `Il2CppSprocket.X`. **Use `Sprocket.X`**.
- Fields show up as properties (`get_x` / `set_x`), and you use them like properties: `mode.vehicleSpawns`.
- Compiler-made classes get plain names: the state machine of `async Load()`, `<Load>d__38`, appears as `_Load_d__38`. Seeing one tells you the method is `async` (see chapter 5 on why that matters).
- Enums keep their values; the kit's tools (or a decompiler) show them.

Finding the right class

1. **Start from what you see.** The editor's right-hand panel for cannons? Search `~CannonEditor`. The custom battle screen? `~CustomBattle`. The main menu? `~MainMenu`, `~MenuPanel`.
2. **Look at the scene.** `Player.log` prints scene changes ("Scene transition from 'MainMenu' to 'Defence'"), and UnityExplorer shows the live objects and the components on them, which are classes you can search for.
3. **Follow the fields.** Once you have one object, its fields lead to the rest. Example from the Battle Editor:
`DeathmatchGameMode` → `context.SetupContext.SetupConfig` → a `BattleConfig` → `Teams[i].Units`.
4. **Prove it with a log line** before building on it (chapter 5).

Classes the kit's mods use (Sprocket 0.2.55.5)

Class	Used for
<code>Sprocket.VehicleDesigner.VehicleDesignerCore</code>	The vehicle editor: the open design, loading a blueprint.
<code>Sprocket.Vehicles.VehicleBlueprintSerializer</code>	Design ↔ blueprint JSON (<code>ToBlueprint</code> , <code>SerializeToJSON</code> , <code>DeserializeJSON</code>).
Part editors, e.g. <code>...Cannons.Editor.CannonEditor</code> , <code>...PlateStructures.Design.PlateStructureEditor</code>	The right-hand panels; <code>OnGUI(IGUILayout layout)</code> draws them.
<code>Sprocket.UI.IGUIElementDrawer</code>	Drawing in those panels: <code>Header</code> , <code>InfoField</code> , <code>Button</code> , <code>ToggleField</code> .

<code>Sprocket.MenuPanel</code>	Main-menu style button lists: <code>Button(label, UnityAction, interactable)</code> .
<code>Sprocket.DeathmatchGameMode</code>	A running custom battle: <code>vehicleSpawns</code> , <code>context</code> , <code>RetryAsyncVoid()</code> .
<code>Sprocket.CustomBattles.BattleConfig</code> , <code>TeamDefinition</code> , <code>UnitDefinition</code>	A custom battle's map, teams and tanks.
<code>Sprocket.Vehicles.IVehicleGateway</code>	A vehicle: its <code>transform</code> , <code>gameObject</code> , design info.

5 Changing the game with Harmony patches

A patch is your method that Harmony runs before (**prefix**) or after (**postfix**) one of the game's methods. It's how a mod changes what the game does without touching its files.

The shapes of a patch

```
[HarmonyPatch]                                // this class holds patches
static class MyPatches
{
    // After the game draws the Cannon panel: add a line to it.
    [HarmonyPostfix, HarmonyPatch(typeof(CannonEditor), nameof(CannonEditor.OnGUI))]
    static void AfterPanel(CannonEditor __instance, IGUILayout layout) { ... }

    // Before the game ends a battle: return false to stop it (the game's method is skipped).
    [HarmonyPrefix, HarmonyPatch(typeof(DeathmatchGameMode), nameof(DeathmatchGameMode.CompleteMission))]
    static bool BeforeWin(DeathmatchGameMode __instance) => !keepBattleGoing;

    // A method with several versions (overloads): name the parameter types.
    [HarmonyPrefix, HarmonyPatch(typeof(VehicleMaterialPainter), nameof(VehicleMaterialPainter.Process),
        new[] { typeof(VehicleMaterial) })]
    static void BeforePaint(VehicleMaterialPainter __instance, VehicleMaterial material) { ... }
}
```

In your patch method	Gets you
<code>__instance</code>	The object the game's method was called on.
A parameter with the same name as the game's	That argument (names must match exactly; types must match).
<code>__result</code> (postfix; ref to change it)	The game method's return value.
Prefix returns <code>bool</code>	<code>false</code> skips the game's method (and the other prefixes after yours).

You can leave out any parameter you don't need. Apply the patches in `Load()` , one class at a time:

```
var harmony = new Harmony("yourname.sprocket.mymod");
foreach (var feature in new[] { typeof(PanelPatches), typeof(BattlePatches) })
    try { harmony.PatchAll(feature); }
    catch (Exception ex) { Log.LogError($"{feature.Name} could not attach: {ex.Message}"); }
```

Rule 1: prove the patch runs

A patch that attaches without an error is **not** proof that it runs. Log from inside it the first time it fires. In a native (IL2CPP) build, the compiler often copies small methods into their callers ("inlining"). Calls that were inlined never reach the method you patched, so your patch silently does nothing.

Real examples

- The Battle Editor first patched `DeathmatchGameMode.LoadGameSetup` and `Load` (both `async`, so their bodies are tiny starter stubs). The patches attached, but never ran once. The fix was to stop hooking and change the battle's own data instead: edit the custom battle's team list, then call the game's own `Retry`.
- The Sprocket Tweaks mod found that `EngineRules.CalculateMass` is merged into its callers, so hooking it alone did nothing. It hooks `VehicleComponent.SetMass` instead, which every part goes through.

Good places to patch: methods the game calls through an interface or as virtual methods (`OnGUI`, `Update`-like methods, event handlers), methods passed as callbacks (menu buttons), and bigger methods that every path goes through. **Risky:** tiny getters, `async` methods, and helpers that are only called from one place.

Rule 2: never let an exception escape

An exception thrown from your patch lands in the game's native code, and can crash it. Wrap every patch body:

```
static void AfterPanel(CannonEditor __instance, IGUILayout layout)
{
    try { /* your code */ }
    catch (Exception ex) { Plugin.ModLog.LogError(ex); }
}
```

Rule 3: be careful with small structs passed by value

The Battle Editor's first version patched `VehicleSpawner.Spawn(..., CancellationToken)`. The game crashed the moment a battle spawned its tanks, and the crash stopped when that one patch was removed. Methods taking small `struct` parameters by value (a `CancellationToken` is one pointer inside a struct) are the likely trouble for the IL2CPP patching layer. If a patch like that crashes, patch another method, or do the work before or after the call instead.

Rule 4: keep patches light

Some game methods run thousands of times: the painter paints every material of every part when a tank loads. Quality of Life's paint patch reads its part list **once per frame** and reuses it, not on every call. Heavy work (loading, saving, rebuilding a design) never goes in a patch or in `OnGUI`: queue it, and do it from a `MonoBehaviour's Update()` (chapter 6).

6 Running your own code every frame

Patches react to the game. For your own behaviour (a key press, a tool, an editor), you need a **MonoBehaviour**: a Unity script whose methods Unity calls every frame.

Adding one

```
public sealed class MyTool : MonoBehaviour
{
    public MyTool(IntPtr pointer) : base(pointer) { }    // required for every injected MonoBehaviour

    public void Update()    { /* every frame: read keys, move things, run queued work */ }
    public void OnGUI()    { /* draw on screen (called several times a frame) */ }
}

// in Plugin.Load():
AddComponent<MyTool>();
```

- `AddComponent<T>()` registers your class with the game's runtime and attaches it to a `BepInEx` object that survives scene changes, so one instance lives from the main menu to every battle.
- The log line `Registered mono type YourMod.MyTool in il2cpp domain` means it worked.
- You may also see warnings like *"Method ... has unsupported parameter ..."* for your own helper methods. They're harmless: it only means the game can't call those methods, and it doesn't need to. Keep Unity's own methods (`Update`, `OnGUI` ...) without parameters.

Reading keys and the mouse

Sprocket uses Unity's **Input System**. Read devices directly; it always works, even when the game's own controls are off:

```
var keys = Keyboard.current; var mouse = Mouse.current;
if (keys.f9Key.wasPressedThisFrame) Toggle();           // pressed this frame
if (keys.shiftKey.isPressed) speed *= 2;                // held down
Vector2 at = mouse.position.ReadValue();                 // pixels, from the BOTTOM-left
Vector2 moved = mouse.delta.ReadValue();                 // movement this frame
float wheel = mouse.scroll.ReadValue().y;               // wheel
if (mouse.leftButton.wasPressedThisFrame) Click(at);
```

Screen corners

The mouse counts from the **bottom**-left corner; on-screen GUI rectangles count from the **top**-left. Convert with `new Vector2(at.x, Screen.height - at.y)`.

Time and pausing

- `Time.timeScale = 0` freezes the game (physics, AI, animations) but not your `Update`. The Battle Editor freezes the battle this way while you edit, and puts the old value back after.
- While frozen, use `Time.unscaledTime` and `Time.unscaledDeltaTime`: the normal ones stop.
- Frozen also means **no physics steps**: objects you move aren't where raycasts look for them until you call `Physics.SyncTransforms()` (see chapter 13).

Finding game objects

```
var mode = Object.FindObjectOfType<DeathmatchGameMode>(); // null outside a custom battle
```

`FindObjectOfType` searches the whole scene, so it's slow. Don't call it every frame for something that doesn't change: find it once and keep it, or search only while you're waiting for it to appear.

7 Working with game objects (IL2CPP rules)

Game objects in your code are .NET wrappers around objects that really live in the native game. They behave like normal C# objects, but with a few rules worth learning early.

Topic	Do this	Not this
Casting	<code>obj.TryCast<T>()</code> (null if it isn't a <code>T</code>) or <code>obj.Cast<T>()</code> . Works for interfaces too: <code>spawn.Vehicle.TryCast<IVehicleGateway>()</code>	<code>obj is T</code> , <code>obj as T</code> , <code>(T)obj</code> : they check the wrapper, not the game object.
Game lists	<code>Il2CppTypeSystem.Collections.Generic.List<T></code> : loop with <code>for (int i = 0; i < list.Count; i++) list[i]</code> .	LINQ directly on game lists: copy to a normal list first.
Game arrays	<code>Il2CppTypeReferenceArray<T></code> (objects), <code>Il2CppTypeStructArray<T></code> (numbers, structs). Make one from a C# array: <code>new Il2CppTypeReferenceArray<T>(array)</code> .	
Callbacks	<code>DelegateSupport.ConvertDelegate<UnityAction>(new Action(...))</code> , and keep it in a field as long as the game might call it.	A temporary delegate: .NET can clean it up while the game still holds it, and the next call crashes.
Same object?	Compare <code>a.Pointer == b.Pointer</code> (the native address), or Unity's <code>GetInstanceID()</code> .	<code>ReferenceEquals</code> : two wrappers can wrap one game object.
Destroyed Unity objects	<code>if (obj == null)</code> : Unity's <code>==</code> knows about destroyed objects.	<code>obj?.x</code> or <code>obj ?? y</code> on Unity objects: they miss destroyed ones.
Your own component types	<code>AddComponent<T>()</code> from the plugin registers <code>T</code> for you.	
Strings, numbers, <code>Vector3</code> , <code>Color</code> ...	Just use them: they convert automatically.	

Not every Unity method exists

When the game was built, Unity **removed code the game never used**. The interop files only contain what's left. For example, `GUI.Button`, `GUI.Toggle` and `GUI.TextField` exist in Sprocket, but `GUI.BeginScrollView` doesn't, so the Battle Editor's design list scrolls by hand. Before you plan a feature around a Unity method, check it's in the interop files (the kit's `ApiListner` will tell you). If it isn't, your code won't even compile against it.

Async game methods

Many game methods return a `Task` (loading a design, restarting a battle). Calling one starts the work and returns at once. The work finishes later, over several frames. The simple and safe pattern: call it, then check in `Update()` each

frame until the result is there. The Battle Editor restarts a battle this way, then waits until every new tank reports `Spawned` before it moves them.

8 Adding user interface

There are three ways to put your own controls on screen. Pick by where the player is.

A. Inside the game's own panels (editor)

Best in the vehicle editor: your controls look like the game's, and clicks work normally. Postfix a part editor's `OnGUI` and draw with `IGUIElementDrawer`. From Quality of Life's gun length feature:

```
[HarmonyPostfix, HarmonyPatch(typeof(CannonEditor), nameof(CannonEditor.OnGUI))]  
static void Draw(CannonEditor __instance, IGUILayout layout)  
{  
    try  
    {  
        var gun = __instance.Component?.Blueprint;  
        var ui = layout.TryCast<IGUIElementDrawer>();  
        if (gun == null || ui == null || gun.Caliber == 0) return;  
        ui.Header("Gun length");  
        ui.InfoField($"Barrel L/{gun.BarrelLength / (float)gun.Caliber:0.##}", 1);  
    }  
    catch (Exception ex) { Plugin.ModLog.LogError(ex); }  
}
```

- `InfoField(text, lines)`: give it exactly as many lines as the text has (use `\n`).
- Buttons: `ui.Button(label, unityAction, ref tooltip)` with a kept `UnityAction` (chapter 7).
- The panel only redraws when asked: after a click that changes what's shown, call the editor's `RequestRedraw()`.
- `layout.BeginDropdown(title, expanded, Action<bool>)` makes a section the player can fold away.

B. In the game's menus

Menu screens build their button lists with `MenuPanel.Button(label, onClick, interactable)`. A postfix on it can add your button right after a game button (chapter 3's example; the Battle Editor button works this way). Your button's action can even call the game button's own action, so it opens the game's screen with your mod "armed".

C. Your own overlay (IMGUI)

Anywhere else (in a battle, over the map), draw in your `MonoBehaviour`'s `OnGUI` with `GUI.Box`, `GUI.Label` and `GUI.Button`. It's quick, plain-looking, and has two traps in Sprocket:

Trap 1: in battles, IMGUI buttons never get the click

They draw and highlight, but `GUI.Button` never returns `true`, because the game keeps the mouse events. The fix the Battle Editor uses: draw the buttons as usual, **remember each button's rectangle and action**, and handle the click yourself in `Update()` with `Mouse.current`.

```
readonly List<Rect Area, Action Act> buttons = new();  
  
void Button(Rect area, string text, Action act) { GUI.Button(area, text); buttons.Add((area, act)); }  
  
public void OnGUI() { buttons.Clear(); Button(new Rect(20, 60, 120, 26), "Save", Save); /* ... */ }
```

```
public void Update()
{
    var mouse = Mouse.current;
    if (mouse == null || !mouse.leftButton.wasPressedThisFrame) return;
    var m = mouse.position.ReadValue();
    var p = new Vector2(m.x, Screen.height - m.y); // GUI counts from the top
    foreach (var (area, act) in buttons.ToList())
        if (area.Contains(p)) { act(); return; }
}
```

Trap 2: clicks go through to the game

A click on your overlay also reaches whatever is behind it (the editor, or your own "click the ground" code). Keep the rectangles of your panels, and ignore map clicks while the mouse is over one.

The mouse pointer and the game's controls

- While you're driving a tank, the game **hides and locks the pointer every frame**. To show it, set `Cursor.lockState = CursorLockMode.None` and `Cursor.visible = true` every frame (in `Update` and `OnGUI`, which runs last), and stop when your tool closes.
- The game's controls keep working under your UI (in the Battle Editor, right-dragging to look also aimed the gun and opened the command wheel). Switch them off while your tool is open, and back on after:

```
var on = InputSystem.ListEnabledActions(); // every game control that's on
for (int i = 0; i < on.Count; i++) { on[i].Disable(); switchedOff.Add(on[i]); }
// later: foreach (var a in switchedOff) a.Enable();
```

Your own `Keyboard.current` / `Mouse.current` reads still work while the game's controls are off.

9 Settings, files and logs

Settings the player can change

```
internal static ConfigEntry<float> Brightness = null!;
// in Load():
Brightness = Config.Bind("Lights", "Flashlight brightness", 80f, "Percent of the sun");
// anywhere:
float b = Brightness.Value; Brightness.Value = 120f; // setting it saves the file
```

The file is `BepInEx\config\<your mod id>.cfg`, created on first start with your defaults and descriptions. Quality of Life also uses settings to *remember* things (folded sections, the last brightness used).

Your own files

Put player data next to the game's, under `Documents\My Games\Sprocket\<your folder>`. The Battle Editor saves battles as `Battles\<map>.json`. .NET's `System.Text.Json` works normally in mods. Keep a version number in your file format from day one, so you can read old files after you change it.

```
var documents = Environment.GetFolderPath(Environment.SpecialFolder.MyDocuments);
var folder = Path.Combine(documents, "My Games", "Sprocket", "Battles");
Directory.CreateDirectory(folder);
File.WriteAllText(Path.Combine(folder, map + ".json"), JsonSerializer.Serialize(battle));
```

Logging

Log	What's in it
BepInEx\LogOutput.log	Your Log.LogInfo / LogWarning / LogError lines, plugin loading, patch errors.
... \LocalLow\HD\Sprocket\Player.log	Unity and the game: scene changes, player states ("Entering player state 'Vehicle Control'"), game errors. The previous run is Player-prev.log.
Your own trace file	For crashes: see below.

A crash loses the end of the log

BepInEx writes its log in batches. When the game crashes, the last lines, the ones you need, may never reach the file. While hunting a crash, also write each important step to your own file straight away:

```
static void Trace(string line)
{
    Plugin.ModLog.LogInfo(line);
    var file = Path.Combine(Paths.BepInExRootPath, "MyMod-trace.log");
    try { File.AppendAllText(file, $"{DateTime.Now:HH:mm:ss.fff} {line}\n"); }
    catch (Exception) { }
}
```

Log **events** (a tool opened, a battle restarted with 11 tanks), never every frame. A trace like `placing: moved 11 tanks to where they were placed` tells you more than any debugger in a native game.

10 Data mods: new parts without code

If you only want new parts (a new add-on shape, a new part to place), you may not need code at all.

Where the game reads parts

File	Holds
Sprocket_Data\StreamingAssets\Parts\<name>Part.json	The part: id, tags, its components (for a structure, its mesh).
Sprocket_Data\StreamingAssets\Localization\en-UK\Parts\<name>.xml	The name players see in the editor.

Parts\roundAddon16PlateStructurePart.json (start of the file)

```
{
  "v": "0.0",
  "guid": "3f1c6a9e-7b2d-4e58-9c14-2a6b8d0e4f16", // the part's id: new, unique, never changes
  "name": "roundAddon16PlateStructure",
  "tags": [ "plateStructurePrefab" ], // where it appears (the Addon Structures palette)
  "components": [ { "type": "plateStructurePrefab",
    "info": { "mesh": { "smoothAngle": 23.5, "format": "freeform",
      "mesh": { "vertices": [ 0.125, 0, 0, ... ] ... } } } } ]
}
```

Making one

- 1 **Copy a game part** that's close to what you want (the add-on cube, for example) into your mod's own `Sprocket_Data\StreamingAssets\Parts\` folder, under a new file name.
- 2 **Give it a new guid** (any GUID generator, e.g. PowerShell's `[guid]::NewGuid()`) and a new `name`. Never reuse the game's ids.
- 3 **Change the data**: the mesh, the size. For generated shapes, write a small program; the kit's `tools\RoundAddonParts` generates the round add-ons.
- 4 **Add the name file** in `Localization\en-UK\Parts\`, copied from the original part's.
- 5 **Install**: copy your `Sprocket_Data` folder into the game folder, or add the folder as a mod in Mod Manager.

The id is forever

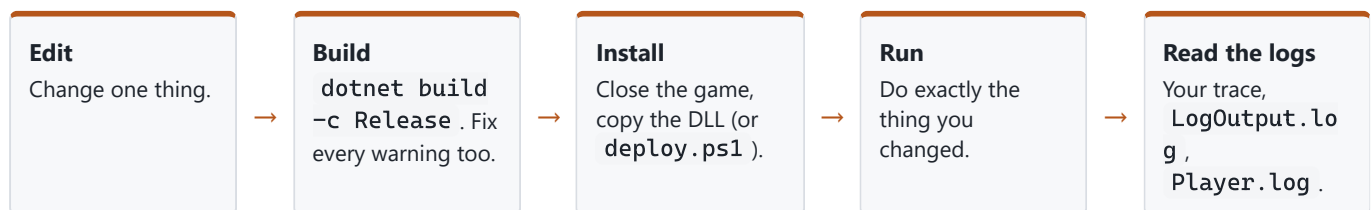
Saved designs remember each part by its `guid`. Change it (or remove the part) and every design that used it stops loading that part. Keep ids stable across versions.

Blueprint files, in one paragraph

`.blueprint` files are JSON. `header` has the name, mass and version. `objects` lists the parts: `guid` is the part type (from the Parts files), `vuid` / `pvuid` are its id and its parent's id, and `transform` has the position, rotation (Unity Euler angles plus a 4th "placement spin" value) and scale. `blueprints` holds settings blocks, referenced by `BlueprintVuid`, and `meshes` holds hand-made shapes. Code mods convert between a design and this JSON with `VehicleBlueprintSerializer`, the same way the game saves.

11 Testing and debugging

The loop



Change **one thing at a time**. When five changes go in together and the game crashes, you don't know which one did it.

Test without the game

Starting Sprocket takes time, so keep logic that doesn't need the game (geometry, file formats, maths) in files that don't use Unity or game classes. Then a plain console project can include those same files and test them in seconds. The kit's `SprocketQoL.Tests` does this: it links Quality of Life's mesh-cutting and conversion code and runs it on real saved designs, with no game running.

```
<ItemGroup>    <!-- in the test project: use the mod's own source file -->
    <Compile Include="..\SprocketQoL\MeshCut.cs" Link="MeshCut.cs" />
</ItemGroup>
```

Looking inside the running game

- **UnityExplorer**: browse scenes, objects and components live; see values change; find the class behind anything on screen.
- **Log what you find**: counts, names, positions. For example, "*editor open on 'Defence': 1 of the battle's tanks moved away, 59 game controls, 5 HUD canvases, 1 fog off*" told us at once what the editor had switched off.
- **Screenshots** of what's wrong, with the log lines from the same moment, are the fastest way to explain a bug to yourself or anyone else.

When the game crashes

1. Read the end of `Player.log` (or `Player-prev.log` after restarting): the last scene and player state tell you *when* it happened.
2. Read your trace file: the last step your mod reached.
3. Turn patches off one class at a time until the crash stops. The last one you turned off is where to look.
4. Suspect, in this order: patches on methods with struct parameters, exceptions escaping a patch, and objects used after the game destroyed them.

12 Packaging and sharing

Before each release

- Raise the version in `[BepInPlugin(...)]`. Keep the **id** the same forever.
- Build `-c Release`, and test that build in the game, not an older one.
- Check your DLL doesn't contain your folder paths: the kit's `PathMap` setting does this. You can check with `findstr /c:"YourWindowsName" YourMod.dll`, which should find nothing.

The zip

Lay the zip out **like the game folder**, so it can be unzipped straight into it, and so Mod Manager knows where each file goes:

```
YourMod-1.0.0.zip
├─ BepInEx
│   └─ plugins
│       ├── YourMod.dll
│       ├── YourMod-README.txt    ← inside plugins, not at the zip's root
│       └── YourMod-LICENSE.txt
```

A README at the zip's root would be copied into the game's own folder. A data mod's zip holds its `Sprocket_Data\...` tree instead.

What not to ship

- **No game files**, no `BepInEx\interop` DLLs (they're made from the game's code, and every player makes their own), no copy of BepInEx itself.
- **No changed game files**. Mods change the running game, never `GameAssembly.dll` or other game files on disk.

Publishing

- A GitHub repository with a release per version is the easy way: players download the zip, and you keep the source.

- Say which **Sprocket version** and **loader** you tested with. Release untested features as a *pre-release* until you've tried them in the game.
- Pick a licence. The kit uses MIT: anyone can use and change the code if they keep the notice.
- Write the README for players: what each feature does, where to click, and how to undo it.

When Sprocket updates

The loader rebuilds `BepInEx\interop` for the new game version. Rebuild your mod against it: if a class or method was renamed, the build tells you where. Then check in the game that each patch still *fires* (your log lines), because a method can still exist but no longer be called, or be inlined now.

For players

A code mod runs with the same rights as the game, so it can do anything a program on your PC can. Only install DLL mods from people you trust, ideally with their source code published.

13 Pitfalls we hit, and the fixes

Every row here cost at least one test run. Each came from building Quality of Life or the Battle Editor.

Symptom	Cause	Fix
Patch attaches, but nothing happens	The method was inlined into its callers (small or <code>async</code> methods).	Log inside every patch. Patch a method every caller goes through, or change the game's data instead.
Game crashes as soon as a patched method runs	A patch on a method with a small struct parameter passed by value (<code>CancellationToken</code>).	Remove that patch; do the work before or after the call.
Crash, no clue in the log	BepInEx writes its log in batches; the end is lost.	Your own trace file, written line by line.
ImGui buttons draw but don't click (in battle)	The game keeps the mouse events.	Record button rectangles; click them yourself from <code>Mouse.current</code> .
Clicks on your overlay also do things in the game	ImGui doesn't stop the click reaching the game.	Skip map clicks while the mouse is over your panels.
Mouse pointer invisible	The game hides and locks it every frame while driving.	Show and unlock it every frame while your tool is open.
Right-drag in your tool also aims, fires, opens menus	The game's controls are still on.	Disable every enabled <code>InputAction</code> ; enable them again on leaving.
Your camera moves snap back	The game camera is moved every frame by a rig following the tank.	Use your own camera: <code>CopyFrom(gameCamera)</code> and copy its <code>HDAAdditionalCameraData</code> (<code>CopyTo</code>); switch the game's off.
Objects you create are pink	A new object's default material uses a shader HDRP can't draw.	A shader the game ships, e.g. <code>Shader.Find("HDRP/Unlit")</code> (check it isn't null), colour in <code>_UnlitColor</code> . One material per colour, not one per object.

Clicking your objects misses them while the game is frozen	<code>Time.timeScale = 0</code> : no physics step, colliders stay where they were made.	<code>Physics.SyncTransforms()</code> after moving them.
A hidden tank still shows its tracks, and the log fills with audio warnings	A central updater keeps driving tracks and sound of switched-off tanks.	Move the tank far out of sight while hidden, and put it back after.
Extra directional lights do nothing	HDRP draws only one directional light (the sun).	Use point lights for extra light (Quality of Life's fullbright does).
Fog hides the ground from high up	The map's HDRP fog volume.	Set the volume's <code>Fog</code> component <code>active = false</code> while needed; put it back after.
A Unity method "doesn't exist"	Stripped from the game build because the game never used it.	Check with <code>ApiListner</code> ; build it from methods that exist (e.g. scroll a list by hand).
Game screen refuses to start ("Player team has no units")	The game's own checks still apply to screens you reuse.	Fill in what it checks (the Battle Editor adds a placeholder tank), or don't reuse that screen.
Copying the DLL fails	The running game locks mod DLLs.	Close the game first; a tiny leftover <code>Sprocket.exe</code> (under 1 MB) isn't a running game.
Things from the last battle reappear	Your component and static fields live as long as the game runs, across battles.	Reset per battle: remember which battle (<code>GetInstanceID()</code>) your data belongs to.

14 Case study: how the Battle Editor is built

The Battle Editor lets you open a map, place any tank for either team, set its facing and who drives it, and play the battle as placed. It's about 1,000 lines, and it uses almost everything in this guide. Its source is in its own repository, apart from the kit.

Files

File	Job	Techniques
<code>Plugin.cs</code>	Starts everything: adds the editor component and applies two patch classes, each in its own <code>try</code> .	Ch. 3, 5
<code>Menu.cs</code>	Adds a Battle Editor button under the game's Custom Battle button. Clicking it runs the game's own Custom Battle action and "arms" the editor.	Postfix on <code>MenuPanel.Button</code> ; kept <code>UnityAction</code>
<code>BattleEditor.cs</code>	The editor itself: opens when the battle's tanks are in, freezes the battle, gives you a camera, markers, panels and keys; puts everything back on leaving.	<code>MonoBehaviour</code> , Input System, own camera, <code>IMGUI</code> with self-handled clicks
<code>Battle.cs</code>	Plays the battle: rewrites the custom battle's team list with your tanks, restarts it, then moves each new tank to its place. Holds back win or loss when a team is empty.	Changing game data, async wait in <code>Update</code> , prefix returning <code>false</code>
<code>BattleFile.cs</code>	The battle file (JSON): map, and per tank an id, team, design, position, facing, who drives it, and orders.	Plain C#, tested without the game

What happens when you use it

- 1 **Main menu → Battle Editor.** The postfix added the button; its action sets `EditNext` and invokes the game's Custom Battle action. The game's own screen opens: you pick the map there. An empty team gets a placeholder tank, because the screen won't start without one.
- 2 **The battle loads.** Each frame, the editor checks for a `DeathmatchGameMode` whose `vehicleSpawns` are all `Spawned`, then opens.
- 3 **Opening** saves everything it will change, then: `Time.timeScale = 0`; its own camera (copied from the game's), looking straight down; the game's controls, HUD and fog off; the battle's tanks moved 1 km under the map; the mouse pointer shown every frame.
- 4 **Editing.** Clicks are read from `Mouse.current`: on a panel, they run the button under the pointer; on the map, they place or pick a tank (raycast against the ground and the marker boxes). The markers use an HDRP unlit material, and `Physics.SyncTransforms()` keeps them clickable while frozen.
- 5 **Play.** Save the file, put everything back, write your tanks into `BattleConfig.Teams[t].Units`, call the game's `RetryAsyncVoid()`. Then, each frame: once all *new* spawns report `Spawned`, move each tank to its place and zero its speed.

Decisions worth copying

- **Use the game's own flows.** The editor doesn't load maps or spawn tanks itself: the game's Custom Battle screen and its Retry do it. The mod only changes the inputs and fixes up the result.
- **Data over hooks.** When hooks on loading didn't fire, changing the battle's data (its team list) did the job with no hook at all.
- **Put back exactly what you changed.** Every change on opening (time scale, camera, controls, HUD, fog, tanks, cursor) is remembered and undone on leaving.
- **Trace every step.** The trace file made each test run readable: `play 'Defence': 11 tanks in 2 teams, restarting → placing: moved 11 tanks to where they were placed`.
- **Build for later.** Each tank's control is stored as `"player"`, `"ai"` or `"slot:N"`, so future multiplayer slots fit in the same file format.

A Cheat sheet

Plugin

```
[BepInPlugin("me.sprocket.mod",
    "My Mod", "1.0.0")]
public sealed class Plugin : BasePlugin
{
    public override void Load()
    {
        AddComponent<MyTool>();
        new Harmony("me.sprocket.mod")
            .PatchAll(typeof(MyPatches));
    }
}
```

Postfix / prefix

```
[HarmonyPostfix, HarmonyPatch(typeof(T),
    nameof(T.Method))]
static void After(T __instance) { }

[HarmonyPrefix, HarmonyPatch(typeof(T),
    nameof(T.Method))]
static bool Before() => true; // false skips
```

MonoBehaviour

```
public sealed class MyTool : MonoBehaviour
{
    public MyTool(IntPtr p) : base(p) { }
    public void Update() { }
    public void OnGUI() { }
}
```

Setting

```
var e = Config.Bind("Section", "Key",
    5f, "Description");
float v = e.Value;
```

Input

```
var k = Keyboard.current;
k.f9Key.wasPressedThisFrame
k.shiftKey.isPressed
var m = Mouse.current;
m.position.ReadValue() // from bottom
m.leftButton.wasPressedThisFrame
m.scroll.ReadValue().y
```

Game objects

```
x.TryCast<T>() // null if not T
x.Cast<T>()
x.Pointer // identity
for (int i = 0; i < list.Count; i++)
    Object.FindObjectOfType<T>()
```

Callback for the game

```
static UnityAction? a; // keep it!
a ??= DelegateSupport
    .ConvertDelegate<UnityAction>(
        new Action(OnClick));
```

Visible, clickable objects (HDRP)

```
var m = new Material(
    Shader.Find("HDRP/Unlit"));
m.SetColor("_UnlitColor", Color.red);
renderer.sharedMaterial = m;
Physics.SyncTransforms(); // if frozen
```

Build and install

```
dotnet build -c Release
.\tools\deploy.ps1 MyMod
```

B Troubleshooting

Problem	Check
Build: "has no working BepInEx 6 IL2CPP"	<code>GameDir</code> points at a folder without <code>BepInEx\core</code> or <code>BepInEx\interop</code> . Install the loader there and start the game once, or build with <code>-p:GameDir="..."</code> .
Build: type or namespace not found	Use the <code>Sprocket.X</code> name, not <code>IL2CppSprocket.X</code> ; add the <code>using</code> for its namespace (Apilister shows it); the class may be renamed after a game update.
The mod isn't in <code>LogOutput.log</code> at all	Is the DLL in <code>BepInEx\plugins</code> (or enabled in Mod Manager)? Built for <code>net6.0</code> ? Does the class have <code>[BepInPlugin]</code> and inherit <code>BasePlugin</code> ? Is the id the same as another mod's?
"Loading [My Mod]" appears, then an error	An exception in <code>Load()</code> : the log shows it. Keep <code>Load()</code> to setup only; the game world doesn't exist yet.
A patch never runs	Chapter 5, rule 1: inlined or never called. Try another method along the same path, or change the game's data.
"could not attach" when patching	The method name or overload doesn't match (use <code>nameof</code> and the parameter types); a patch parameter's name doesn't match the game's.
Update/OnGUI never runs	Did you call <code>AddComponent<T>()</code> ? Does the class have the <code>(IntPtr)</code> constructor? Look for "Registered mono type" in the log.
Keys do nothing	Read <code>Keyboard.current</code> (the game uses the Input System, not the old <code>Input</code> class); check it isn't null.
My buttons don't respond	In a battle, ImGui clicks never arrive: handle clicks yourself (chapter 8).
The game freezes at the main menu with BepInEx	The loader build doesn't run on this Sprocket version (chapter 2). Try a build other Sprocket modders confirm.
It worked before a game update	Rebuild against the new <code>interop</code> , then check each patch still logs that it fires.

This guide is part of the Sprocket Mod Kit (MIT licence). Class names are from Sprocket 0.2.55.5. If something here stops matching your game version, the method in chapter 4 finds the new name.